

Rapport technique - Data Lake Finance

Analyse, résultats, justification des choix et captures de preuve

Groupe

Artemiy Smogunov - Nathan Smadja-Tubiana - Patrice Ignongui

Projet Data Lake Finance - EFREI

Lien GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Version générée le 09/07/2026 08:19

Objectif : construire un data lake financier complet, depuis l'ingestion de données brutes jusqu'à l'analyse curated, avec orchestration Airflow, stockage MinIO, indexation Elasticsearch, base PostgreSQL, API FastAPI et contrôles de qualité.

1. Résumé exécutif

Conclusion courte

Le projet fonctionne : les services sont disponibles, le DAG Airflow s'exécute, les données sont ingérées dans les couches raw, staging et curated, et l'endpoint optimisé /ingest_fast réduit fortement le temps d'ingestion. La valeur du projet n'est pas seulement de stocker des données : elle est de rendre chaque étape explicable et vérifiable.

563 Objets raw MinIO	5958 Docs Elasticsearch	5917 Lignes staging
4909 Lignes curated	208 Anomalies	4.24% Taux anomalie

Lecture des résultats : le nombre d'objets MinIO, le nombre de documents Elasticsearch, les lignes staging et les lignes curated ne représentent pas la même granularité. MinIO conserve les preuves brutes, Elasticsearch indexe les observations ticker/date, staging nettoie et enrichit les séries, puis curated ajoute les signaux analytiques et les anomalies.

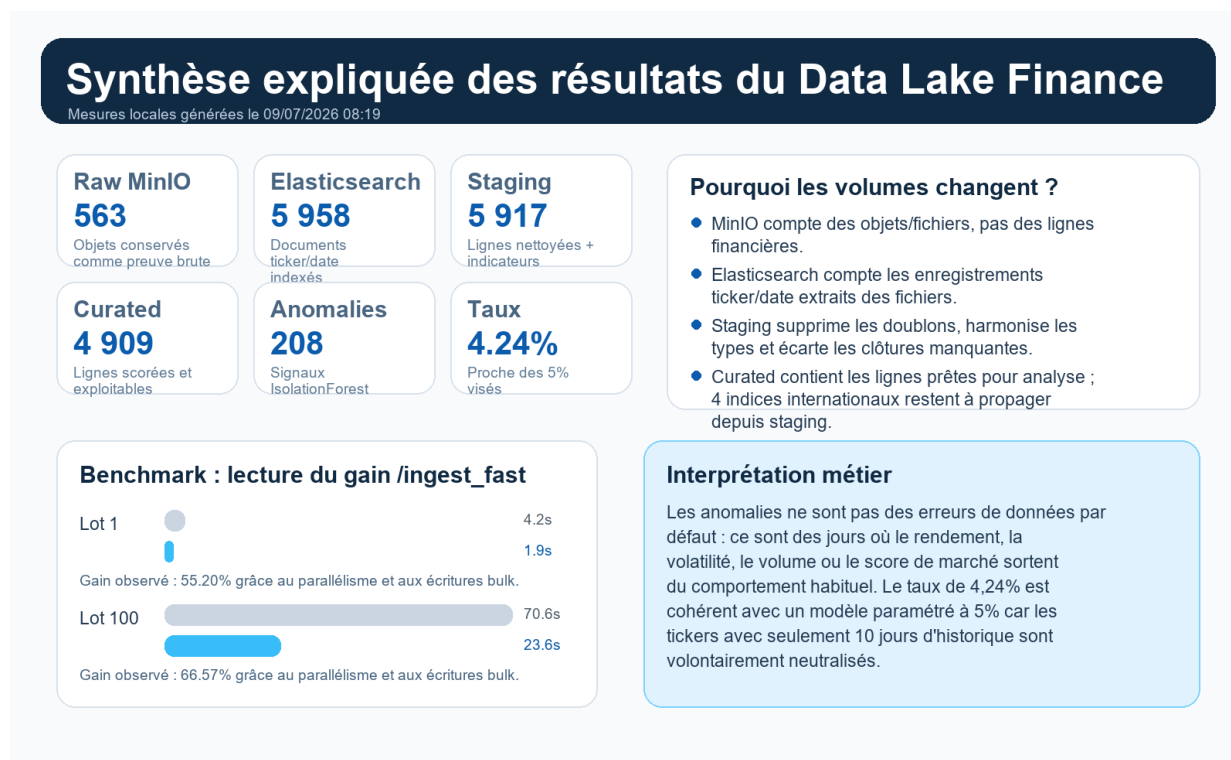
L'état de santé API est ok. PostgreSQL, MinIO, Elasticsearch et Redis répondent correctement ; cela confirme que les résultats du rapport proviennent d'une pile locale opérationnelle.

2. Architecture et intention technique

Couche	Rôle	Pourquoi ce choix
Raw / MinIO	Conserver fichiers et réponses API sans transformation.	Traçabilité : on peut prouver ce qui a été reçu avant nettoyage.
Raw / Elasticsearch	Indexer les observations ticker/date.	Recherche rapide et contrôle de volumétrie par ticker.
Staging / PostgreSQL	Nettoyer, typer, dédupliquer et calculer SMA/EMA/RSI/MACD/Bollinger.	Base relationnelle stable pour transformations reproductibles.
Curated / PostgreSQL	Ajouter score anomalie, type d'anomalie, tendance et signal.	Couche finale directement exploitable par API ou analyse.
Airflow + FastAPI	Orchestrer et exposer le pipeline.	Airflow prouve l'automatisation ; FastAPI permet tests, démos et intégration.

Le choix multi-stockage est volontaire. Un data lake ne force pas toutes les données dans une seule base : il garde le brut dans un stockage objet, rend les données recherchables dans Elasticsearch, puis transforme les données utiles dans PostgreSQL. Cette séparation évite de perdre l'historique brut quand la logique de nettoyage évolue.

3. Résultats observés et explication



Capture de synthèse générée à partir des métriques réelles de l'API /stats et du benchmark.

Pourquoi ces chiffres ?

- 563 objets MinIO : ce sont des fichiers ou payloads bruts. Un objet peut contenir plusieurs lignes financières ; ce volume mesure donc la traçabilité, pas le nombre de cotations.
- 5 958 documents Elasticsearch : ici la granularité est ticker/date, donc le compteur se rapproche du nombre d'observations de marché.
- 5 917 lignes staging : le pipeline supprime les doublons par date, convertit les colonnes numériques et retire les clôtures manquantes. La légère baisse par rapport au raw indexé est attendue.
- 4 909 lignes curated : la couche finale contient les séries scorées. L'écart exact avec staging est expliqué par 4 tickers non propagés en curated (1 008 lignes), principalement ^FCHI, ^FTSE, ^GDAXI, ^N225.

Point de vigilance honnête

L'écart staging -> curated n'est pas une perte silencieuse : les 4 indices internationaux restent visibles en staging. Pour une version industrielle, il faut ajouter ces tickers au périmètre curated ou relancer la transformation curated sur toute la liste staging.

4. Analyse des anomalies

La détection d'anomalies repose sur IsolationForest avec une contamination cible de 5%. Cela signifie que le modèle cherche environ 5% de points atypiques parmi les historiques suffisamment longs. Le résultat observé est 208 anomalies, soit 4.24% des lignes curated.

Ce taux légèrement inférieur à 5% est cohérent : plusieurs tickers courts n'ont qu'environ 10 jours d'historique. Le pipeline les conserve mais neutralise l'apprentissage avancé lorsqu'il n'y a pas assez de contexte statistique, afin d'éviter de produire de faux signaux.

Ticker	Lignes	Anomalies	Taux	Dernière date
AAPL	868	44	5.1 %	2026-07-08
AMZN	263	14	5.3 %	2026-07-08
BRK-B	263	14	5.3 %	2026-07-08
GOOGL	263	14	5.3 %	2026-07-08
JNJ	263	14	5.3 %	2026-07-08
JPM	263	14	5.3 %	2026-07-08
META	263	14	5.3 %	2026-07-08
NVDA	263	14	5.3 %	2026-07-08
^DJI	257	13	5.1 %	2026-07-08
^GSPC	257	13	5.1 %	2026-07-08

Interprétation métier : une anomalie ne signifie pas automatiquement une erreur. Elle signale une journée atypique sur le rendement, la volatilité, le volume ou la combinaison des indicateurs. Pour un analyste financier, ces points servent de shortlist de jours à investiguer : annonces de résultats, choc macroéconomique, changement de tendance, volume exceptionnel ou mouvement brutal de prix.

5. Performance : pourquoi /ingest_fast est plus rapide

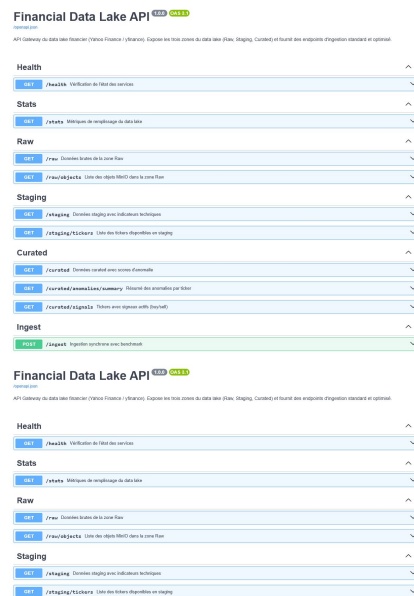
Lot	/ingest	/ingest_fast	Gain
1	4.16 s	1.86 s	55.20 %
100	70.63 s	23.61 s	66.57 %

- L'endpoint standard traite les tickers de façon plus séquentielle ; son temps augmente fortement quand le lot grossit.
- /ingest_fast parallélise les téléchargements yfinance, les écritures MinIO et utilise des écritures bulk vers Elasticsearch.
- Les transformations numériques sont vectorisées avec NumPy/Pandas au lieu de répéter des boucles Python coûteuses.
- Le benchmark a été réalisé avec cache désactivé : le gain observé vient donc réellement de l'architecture, pas d'une réponse déjà mémorisée.
- Le gain passe de 55,20% sur un ticker à 66,57% sur 100 tickers, ce qui montre que l'optimisation est surtout utile à l'échelle batch.

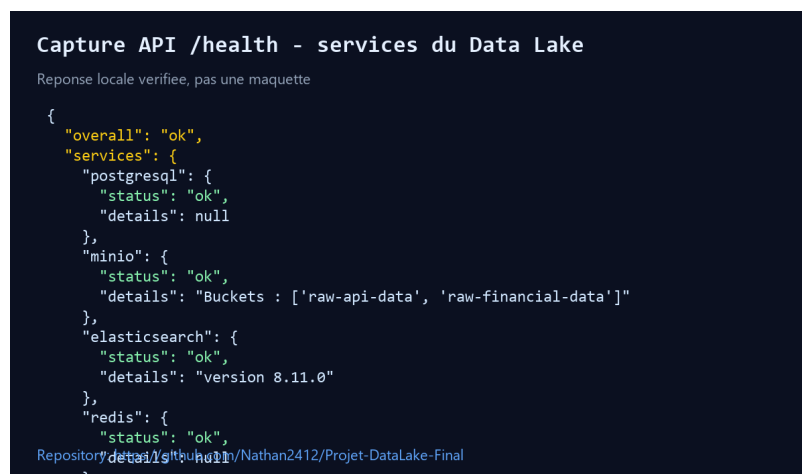
Lecture critique

Le benchmark prouve un gain d'ingestion, pas une vérité absolue sur toutes les charges. Pour aller plus loin, il faudrait répéter les mesures, tester plusieurs périodes et ajouter des percentiles de latence.

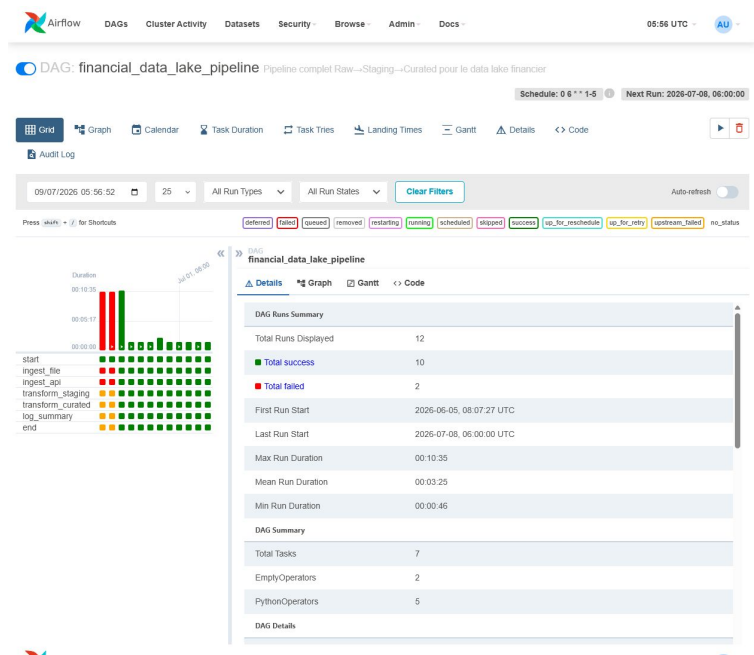
6. Captures de preuve



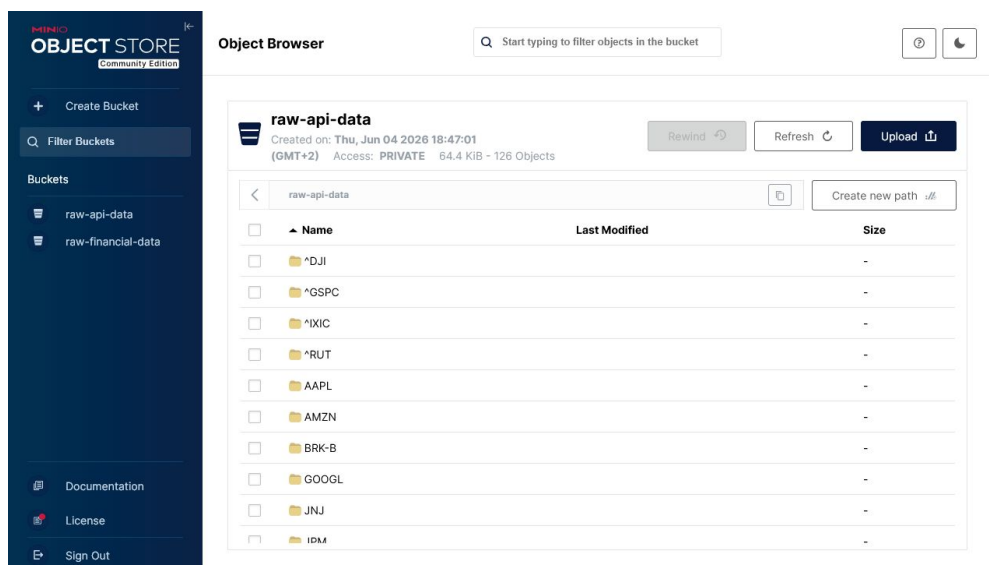
FastAPI Swagger : endpoints d'ingestion, transformation, monitoring et statistiques.



API /health : services PostgreSQL, MinIO, Elasticsearch et Redis disponibles.



Airflow : DAG `financial_data_lake_pipeline` actif et exécutions visibles.



MinIO : buckets `raw-api-data` et `raw-financial-data`, preuve de la couche raw.

7. Limites, améliorations et conclusion

- Propager les 4 indices internationaux manquants de staging vers curated pour supprimer l'écart de 1008 lignes.
- Ajouter une historisation des benchmarks afin de comparer les performances dans le temps.
- Exposer un endpoint de qualité des données indiquant doublons supprimés, valeurs manquantes et tickers ignorés.
- Ajouter des alertes Airflow/Slack ou email sur les échecs DAG, car les anciens échecs restent visibles dans l'interface.
- Compléter l'analyse métier avec des annotations d'événements financiers externes pour expliquer certaines anomalies.

Conclusion : le projet répond au thème finance et aux attendus data lake. Il couvre ingestion, stockage raw, transformation staging, couche curated, orchestration, API, benchmark et preuves visuelles. La réflexion importante est que les résultats ne sont pas seulement des compteurs : ils racontent le passage d'une donnée brute traçable vers une donnée nettoyée, enrichie et interprétable.